

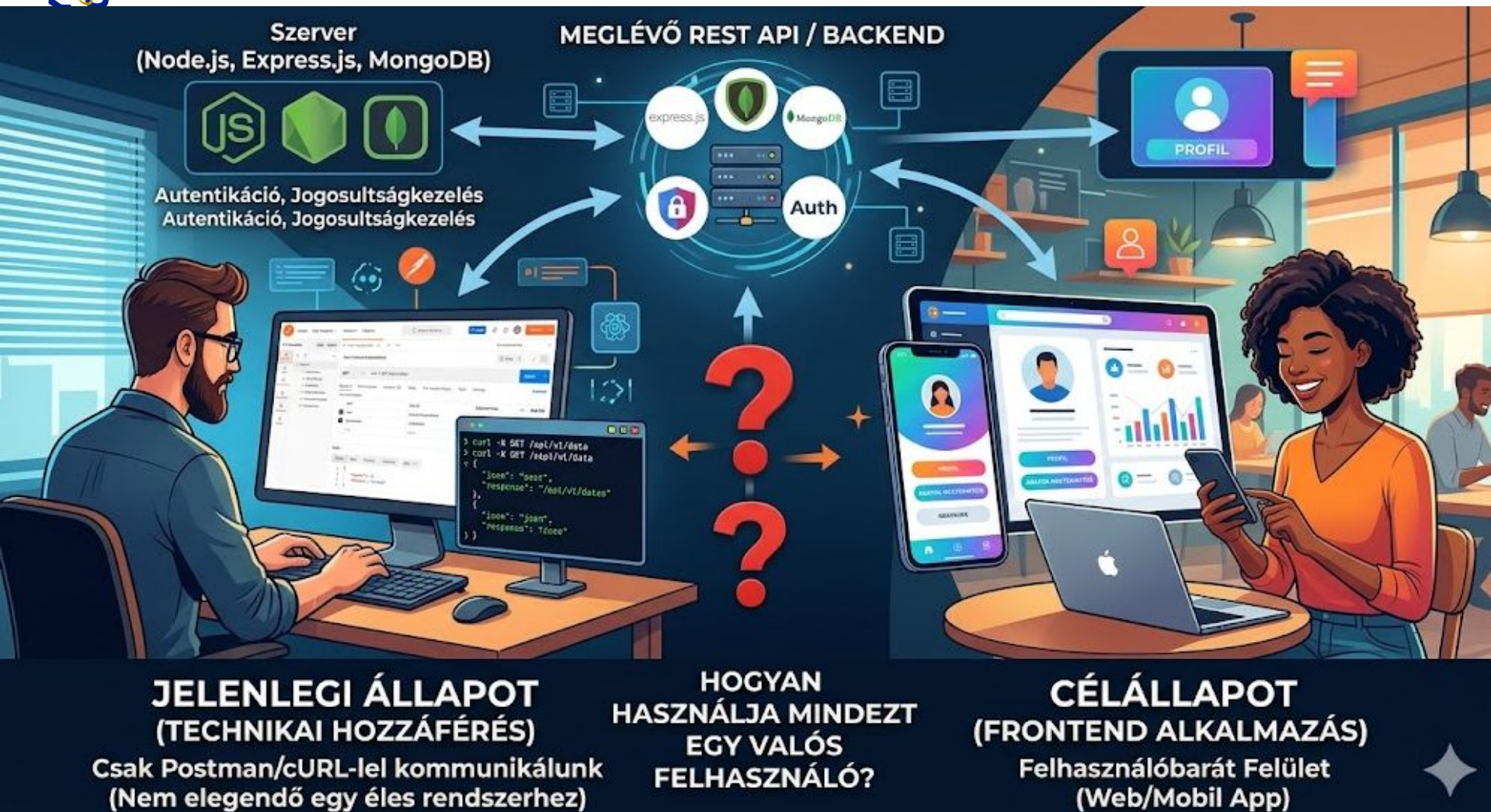


Programrendszerek fejlesztése

8. gyakorlat

Dr. Jánki Zoltán Richárd

Hol tartunk most?



Frontend szerepe a szoftverrendszerben

BACKEND és FRONTEND KAPCSOLATA: A FEJETLEN RENDSZERTŐL AZ INTERAKTÍV ÉLMÉNYIG

BACKEND - FEJETLEN RENDSZER



BACKEND - FEJETLEN RENDSZER

FRONTEND - FELHASZNÁLÓI FELÜLET



FRONTEND - FELHASZNÁLÓI FELÜLET

AZ ÜZLETI LOGIKÁT
EMBERI INTERAKCIÓVÁ
FORDÍTJA

Web-alkalmazások evolúciója

WEBALKALMAZÁSOK EVOLÚCIÓJA: SSR-TŐL SPA-IG

AZ ÁTMENET LÉNYEGE: A MEGJELNÍTÉSI FELELŐSSÉG
A SZERVERRŐL A KLIENSRE KERÜLT.

HAGYOMÁNYOS SSR (Server-Side Rendering)



AJAX KORSZAK (Részleges Frissítések)



SPA CONCEPT (Single Page Application)



SPA előnyök és hátrányok

Előnyök:

- gyorsabb felhasználói élmény
- tisztább szeparáció a frontend és backend között
- a backend újrahasznosítható

Hátrányok:

- hosszabb kezdeti betöltési idő
- SEO nagyobb kihívás
- bonyolultabb állapotkezelés a kliensen



Modern frontend keretrendszerek

- React (2013)
 - sokkal inkább UI könyvtár
 - view réteggel foglalkozik, a többi külső könyvtárakból jön
- Vue.js (2014)
 - progresszív keretrendszer
- Angular (2016)
 - teljes keretrendszer
- TypeScript támogatottság: mindenhol
- Tanulási görbe: Vue → React → Angular



Komponens-alapú gondolkodás

MI AZ A KOMPONENS?

EGY ÖNÁLLÓ, ÚJRAFELHASZNÁLHATÓ UI ÉPÍTŐELEM

EGY KOMPONENS BELSŐ SZERKEZETE

SABLON
(HTML)



STÍLUS
(CSS)

LOGIKA
(TypeScript)



KOMPONENS

A RECEPTGYŰJTEMÉNY ALKALMAZÁS OLDALA

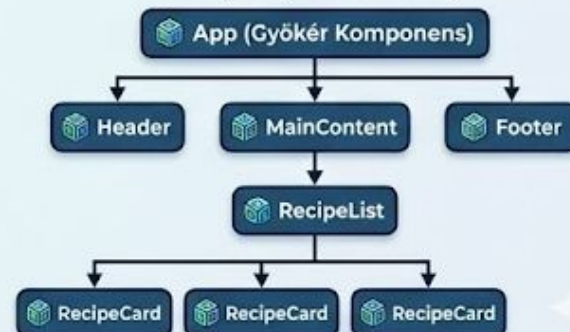


FELBONTÁS
KOMPONENSEKRE

KOMPONENS ALAPÚ ELRENDEZÉS



KOMPONENS FA STRUKTÚRA (Component Tree)



GONDOLKODÁSMÓD

HANEM



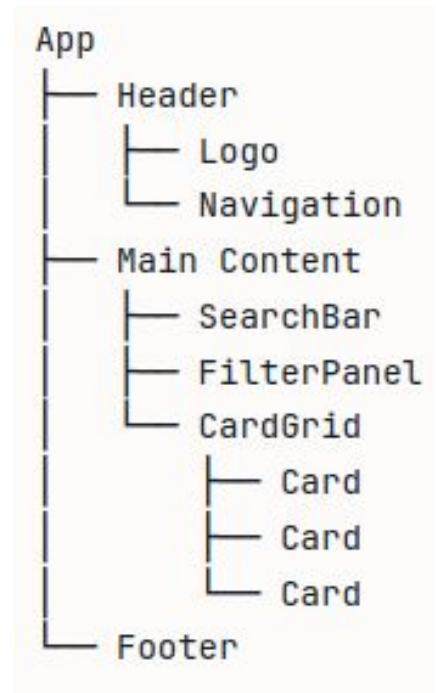
...ÉPÍTŐKOCKÁKAT, AMELYEKBŐL
OLDALAK ÁLLNAK ÖSSZE.



NEM OLDALAKAT
TERVEZÜNK...

Komponens fogalma

- A UI egy önálló újrafelhasználható építőeleme
 - Sablon (template): HTML
 - Logika: JavaScript
 - Stílus: CSS
- Az alkalmazások fa-struktúrát alkotnak
- Alapelvek
 - Egyetlen felelősség
 - Újrafelhasználhatóság
 - Kis méret
 - Egyértelmű interfész
 - Buta vs. okos komponensek



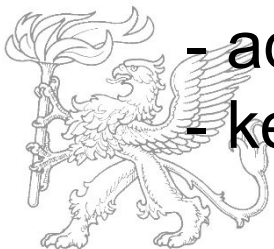
Komponensek közötti kommunikáció

1. Szülő → gyerek
2. Gyerek → szülő
3. Távoli komponensek között

Legjobb gyakorlat: egyirányú adatáramlás (felfelé vagy lefelé)

Anti-pattern:

- azonnali globális store használat
- adatátadás 4-5 szinten keresztül
- kétirányú kötés túlzott használata

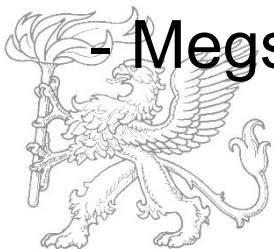


Komponens élethciklus (Lifecycle)

1. Létrehozás
2. Változás-detektálás
3. Frissítés
4. Megsemmisülés

Fontosabb hook-ok (minden keretrendszerben):

- Inicializálás → adatok betöltése
- Bemenet változása → változás figyelése
- Megsemmisülés → cleanup (takarítás)



Tipikus életciklushibák

- Hiányzó cleanup → memory leak
- Rossz hook választás
- Túl gyakori változás detektálás → felesleges renderelés
- Aszinkron műveletek versenye



Komponensekre bontás a gyakorlatban

- 3 fő kérdés
 - Megjelenik-e az elem az alkalmazás több pontján is?
 - Ha igen, akkor önálló komponens.
 - Van-e önálló felelősségi köre?
 - Ha igen, válasszuk le!
 - Kezelhetetlenül nagy fájl lett?
 - Ha 300 sornál hosszabb a kód, valószínűleg tovább bontható.



Gyakorlat

- Készítsük el a frontend alkalmazásunk vázát!
- Generáljuk le a szükséges oldalakat és a navigációkat valósítsuk meg!
- Adjunk hozzá DEMO adatokat!

